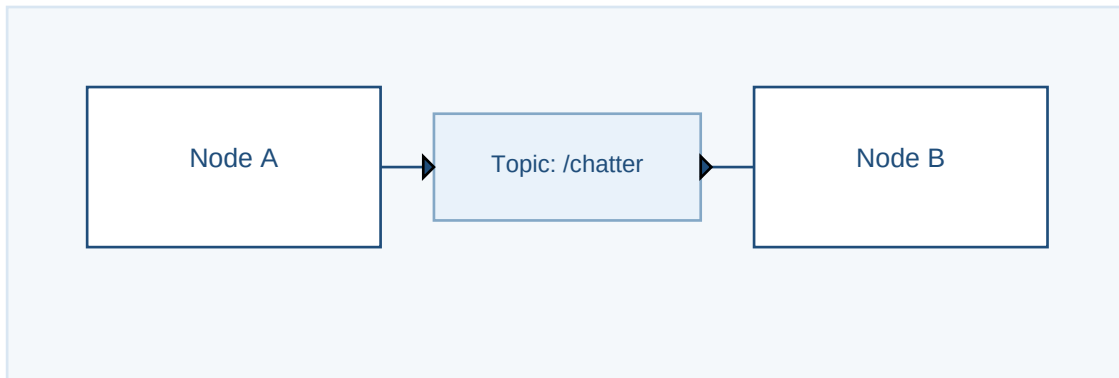


# ROS 2 Comprehensive Guide

Libraries, Classes, Architecture, Syntax, Examples, and Best Practices

2025 Edition

Generated by AI Assistant



This document provides a deeply practical, example-driven tour through ROS 2 (Robot Operating System 2). It covers the key libraries, classes, structure, syntax, and behavioral outcomes with runnable examples. Use this as a quick reference, learning aid, and field guide for building reliable robotic software.

# Table of Contents

Placeholder for table of contents

0

## Introduction

ROS 2 is a modular, distributed framework for building robotic systems. It builds on DDS (Data Distribution Service) and offers publish–subscribe topics, synchronous services, asynchronous actions, parameters, lifecycle management, and a rich tooling ecosystem.

- Core paradigms: nodes, topics, services, actions, parameters, lifecycle
- Language bindings: C++ (rclcpp) and Python (rclpy)
- Transport: DDS vendors (CycloneDDS, Fast DDS, Connex) via rmw layer
- Build and package: ament, colcon, CMake, and package.xml
- CLI tooling: ros2 command for discovery, run, inspect, bag, doctor

## Architecture and Core Concepts

### Nodes and Execution

A node is a process that uses ROS 2 client libraries. Executors spin nodes and dispatch callbacks from subscriptions, timers, and services.

### Topics (Publish–Subscribe)

Topics are asynchronous data streams. Publishers write messages and subscribers receive them. QoS policies control delivery guarantees and history.

### Services (Request–Response)

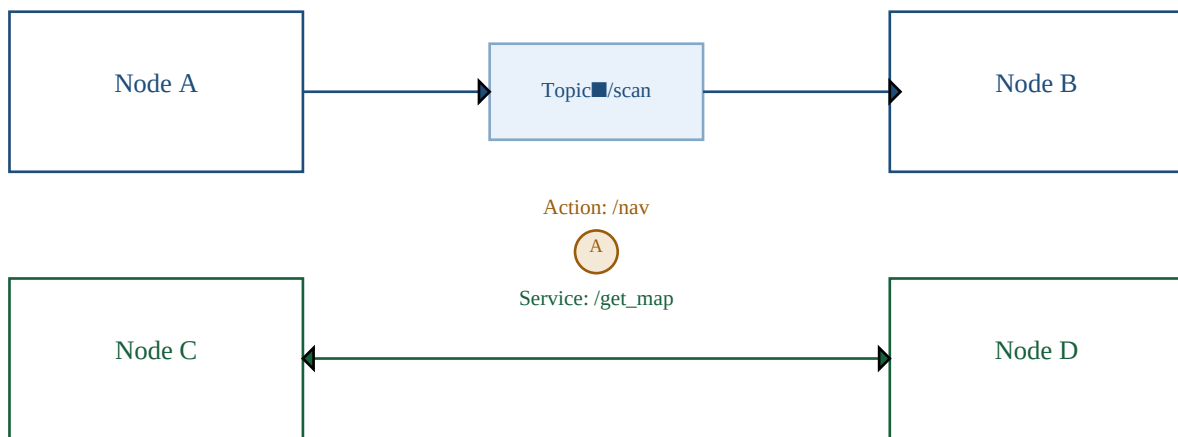
Services provide synchronous request/response RPC semantics between clients and servers.

### Actions (Long-running Goals)

Actions split long-running tasks into goal, feedback, and result channels, allowing cancelation and progress updates.

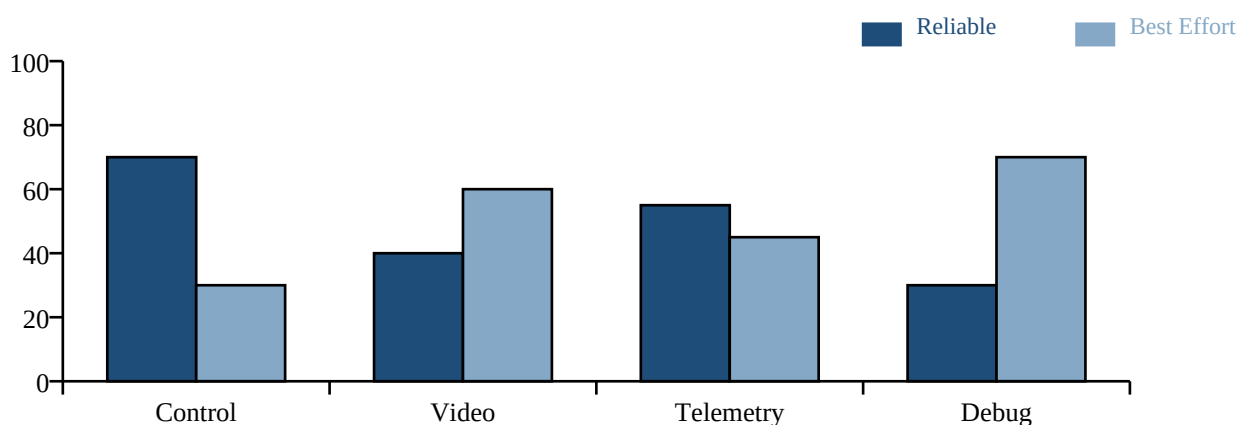
### Parameters and Lifecycle

Parameters are key/value configuration for nodes. Lifecycle nodes have managed states (unconfigured, inactive, active) enabling reliable bring-up.



## Quality of Service (QoS)

| Policy      | Options                    | Effect                                    |
|-------------|----------------------------|-------------------------------------------|
| Reliability | Reliable   Best Effort     | Delivery guarantees vs. speed             |
| Durability  | Volatile   Transient Local | Late-joiners receive last samples with TL |
| History     | Keep Last   Keep All       | How many messages to store in the queue   |
| Depth       | Integer $\geq 0$           | Size of the message queue                 |
| Deadline    | Duration                   | Max allowable period between samples      |
| Lifespan    | Duration                   | Validity period for messages              |
| Liveliness  | Automatic   Manual         | Node presence detection semantics         |



## Client Libraries and Key Classes

### C++ (rclcpp)

| Class                                      | Purpose                                                 | Key API                                           |
|--------------------------------------------|---------------------------------------------------------|---------------------------------------------------|
| rcldcpp::Node                              | Represents a ROS 2 node (parameters, logging, time)     | Node(handle) get_logger(), declare_parameter(),   |
| rcldcpp::Publisher<T>                      | Publishes messages on a topic                           | publish(msg), get_topic_name()                    |
| rcldcpp::Subscription<T>                   | Receives messages from a topic                          | Callback signature: void(const T::SharedPtr&)     |
| rcldcpp::Service<T> / rclcpp::Client<T>    | Service server/client for request/response interactions | create_service<T>(srv, cb), create_client<T>(srv, |
| rcldcpp::action::Server<T> / Client<T>     | Action server/client for long-running tasks             | send_goal(), handle_accepted(), publish_result()  |
| rcldcpp::executors::SingleThreadedExecutor | Executes callbacks on one thread                        | add_node(), spin()                                |
| rcldcpp::executors::MultiThreadedExecutor  | Executes callbacks on a pool of threads                 | add_node(), spin()                                |
| rcldcpp::TimerBase                         | Periodic callbacks                                      | create_wall_timer(period, cb)                     |
| rcldcpp::QoS                               | QoS profiles (reliability, durability, depth)           | QoS(depth).reliable(), keep_last(n)               |
| rcldcpp_lifecycle::LifecycleNode           | Managed lifecycle with states and transitions           | configure(), on_activate(), on_shutdown()         |

## Python (rclpy)

| Class                           | Purpose                                            | Key API                                              |
|---------------------------------|----------------------------------------------------|------------------------------------------------------|
| rclpy.node.Node                 | ROS 2 node abstraction (parameters, logging, time) | Node(name) get_logger(), declare_parameter(),        |
| Publisher                       | Publishes messages on a topic                      | publish(msg), topic_name                             |
| Subscription                    | Receives messages from a topic                     | callback(msg)                                        |
| Client / Service                | Request/response interactions                      | create_client(srv, name), create_service(srv, name)  |
| Timer                           | Periodic callbacks                                 | create_timer(seconds, cb)                            |
| QoSProfile                      | QoS policies for topics                            | QoSProfile(depth=n, reliability=..., durability=...) |
| LifecycleNode (rclpy.lifecycle) | Managed states for reliable bring-up               | on_configure(), on_activate(), on_shutdown()         |

## Examples and Expected Output

### Minimal Publisher (Python)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalPublisher(Node):
    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'chatter', 10)
        self.timer = self.create_timer(0.5, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello ROS 2: {self.count}'
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: {msg.data}')
        self.count += 1

def main():
    rclpy.init()
    node = MinimalPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
```

```
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

*Expected output (console):*

```
[INFO] [minimal_publisher]: Publishing: Hello ROS 2: 0
[INFO] [minimal_publisher]: Publishing: Hello ROS 2: 1
... (repeats)
```

## Minimal Subscriber (Python)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalSubscriber(Node):
    def __init__(self):
        super().__init__('minimal_subscriber')
        self.subscription = self.create_subscription(String, 'chatter', self.listener_callback, 10)
        self.subscription  # prevent unused variable warning

    def listener_callback(self, msg: String):
        self.get_logger().info(f'I heard: {msg.data}')

def main():
    rclpy.init()
    node = MinimalSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

*Expected output (console):*

```
[INFO] [minimal_subscriber]: I heard: Hello ROS 2: 0
[INFO] [minimal_subscriber]: I heard: Hello ROS 2: 1
... (repeats)
```

## Minimal Publisher (C++)

```
#include <chrono>
#include <memory>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>

using namespace std::chrono_literals;

class MinimalPublisher : public rclcpp::Node {
public:
    MinimalPublisher() : Node("minimal_publisher"), count_(0) {
        publisher_ = this->create_publisher<std_msgs::msg::String>("chatter", 10);
        timer_ = this->create_wall_timer(500ms, std::bind(&MinimalPublisher::on_timer, this));
    }
private:
    void on_timer() {
        auto message = std_msgs::msg::String();
        message.data = "Hello ROS 2: " + std::to_string(count_++);
        publisher_>publish(message);
        RCLCPP_INFO(this->get_logger(), "Publishing: %s", message.data.c_str());
    }
}
```

```
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    size_t count_;
};

int main(int argc, char ** argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalPublisher>());
    rclcpp::shutdown();
    return 0;
}
```

## Service (Python): AddTwoInts

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
    def __init__(self):
        super().__init__('add_two_ints_server')
        self.srv = self.create_service(AddTwoInts, 'add_two_ints', self.add)

    def add(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info(f'Req: a={request.a} b={request.b} -> {response.sum}')
        return response

def main():
    rclpy.init()
    node = AddTwoIntsServer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

*Example client call:*

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 2, b: 40}"
```

*Expected output (console):*

```
[INFO] [add_two_ints_server]: Req: a=2 b=40 -> 42
```

## Actions (Conceptual)

Use `rclcpp_action` or `rclpy action` interfaces for long-running goals. Define a Goal, Feedback, and Result type. Clients send goals asynchronously; servers report feedback and results.

## CLI and Build System

### Create a Package

```
# C++ package
ros2 pkg create --build-type ament_cmake --dependencies rclcpp std_msgs my_cpp_pkg

# Python package
ros2 pkg create --build-type ament_python --dependencies rclpy std_msgs my_py_pkg
```

## Build and Run

```
colcon build --symlink-install
source install/setup.bash
ros2 run my_cpp_pkg my_node
ros2 run my_py_pkg my_node
```

## Discovery and Introspection

```
ros2 node list
ros2 topic list
ros2 topic echo /chatter
ros2 interface show std_msgs/msg/String
ros2 doctor --report
```

## Package Structure

### C++ (ament\_cmake)

```
my_cpp_pkg/
  CMakeLists.txt
  package.xml
  src/
    minimal_publisher.cpp
  include/my_cpp_pkg/
    minimal_publisher.hpp
  launch/
    demo.launch.py
```

### Python (ament\_python)

```
my_py_pkg/
  package.xml
  setup.cfg
  setup.py
  resource/
    my_py_pkg
  my_py_pkg/
    __init__.py
    minimal_publisher.py
  launch/
    demo.launch.py
```

## Launch Files

Launch files orchestrate multiple nodes and parameters. Use Python-based launch descriptions for flexibility.

```
# demo.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='my_py_pkg', executable='minimal_publisher', name='talker', parameters=[{'use_sim_time': True}]),
        Node(package='my_py_pkg', executable='minimal_subscriber', name='listener'),
    ])
```

## Testing

Use gtest for C++ and pytest or launch\_testing for Python. Integrate with colcon test.

```
# C++ (gtest) snippet
TEST(TestTopic, Publishes) {
```

```
// Build node, publish, spin, assert received
}

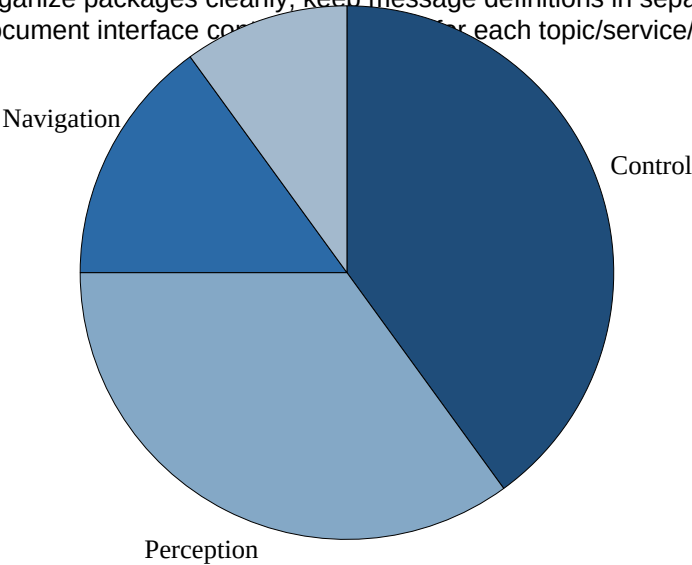
# Python (pytest) example
import pytest

def test_add_two_ints_client():
    assert 2 + 40 == 42

# Run
colcon test --ctest-args -R my_pkg
colcon test-result --verbose
```

## Performance, Reliability, and Best Practices

- Prefer composition (components) to reduce process overhead and enable zero-copy intra-process communication.
- Choose QoS consciously: Reliable for control and critical telemetry; Best Effort for high-rate, lossy streams (e.g., video).
- Use executors appropriate to workload: MultiThreadedExecutor for IO-bound callbacks; SingleThreaded for deterministic order.
- Pin CPU cores and use real-time kernels for hard real-time; avoid dynamic memory in critical loops.
- Leverage loaned messages and intra-process communication to minimize copies (where supported).
- Record and replay with rosbag2 for debugging and regression testing.
- Trace with ros2\_tracing/LTTng and profile callback latencies.
- Secure DDS with DDS-Security (encryption, authentication) when on untrusted networks.
- Use parameters with descriptors, declare early, and validate on set callbacks.
- Organize packages cleanly; keep message definitions in separate interface packages.
- Document interface contracts and provide examples for each topic/service/action.



Typical ROS 2 workload focus per subsystem (illustrative)

## Glossary

| Term      | Definition                                                      |
|-----------|-----------------------------------------------------------------|
| DDS       | Data Distribution Service: the transport layer used by ROS 2    |
| rmw       | ROS middleware abstraction that bridges client libraries to DDS |
| QoS       | Policies influencing message delivery semantics                 |
| Executor  | Runs callbacks for one or more nodes                            |
| Lifecycle | Managed node states for predictable startup/shutdown            |



## Resources and Further Reading

- [ROS 2 Tutorials \(docs.ros.org\)](https://docs.ros.org)
- Design articles and REP documents
- DDS vendor docs (CycloneDDS, Fast DDS)
- [rosbag2](#), [launch](#), [composition](#), and [tracing](#) repositories